

# Martin Sim

916-829-9519 | [martinsim@berkeley.edu](mailto:martinsim@berkeley.edu) | [linkedin.com/in/martinsim](https://www.linkedin.com/in/martinsim) | [github.com/MartinSimulator](https://github.com/MartinSimulator) | [martinsimulator.github.io](https://martinsimulator.github.io)

## Education

### University of California, Berkeley

Expected May 2028

B.S. in Electrical Engineering and Computer Science, GPA: 3.66

Berkeley, CA

- **Relevant Coursework:** Data Structures & Algorithms, Structure and Interpretation of Computer Programs, Foundations of Signals, Dynamical Systems & Information Processing, Linear Algebra & Differential Equations

## Experience

### Software Engineering Intern

June 2026 - Present

Optima

Remote

- Designed Strategy + Registry state machine (Spring Boot) for 12-state claim lifecycle, auto-wiring stateless handlers via dependency injection to replace manual dispatch and cap core service at 330 lines
- Integrated GCP Pub/Sub for asynchronous claim-submission handoff, decoupling state transitions from downstream worker via 3-attempt exponential backoff (50 to 200ms) and dead-letter-queue failover
- Built custom `@OptimisticLockRetry` annotation over Spring Retry to auto-retry JPA version-collision failures on concurrent claim writes, eliminating unhandled 500s across 5 write endpoints
- Closed claim lifecycle end-to-end with three payer/clearinghouse callback endpoints (submission status, adjudication, 835 reconciliation), eliminating all manual database edits needed to advance claims

### Undergraduate Research Software Developer

January 2026 - July 2026

Lawrence Berkeley National Laboratory - ENIGMA (CDSS Data Discovery Program)

Berkeley, CA

- Redesigned field-geochemistry data pipeline around read-only DuckDB engine, deleting 529 lines of ad-hoc per-file parsing and replacing 8 hardcoded data sources with one declarative campaign registry
- Added startup schema-integrity check across DuckDB layer, causing misconfigured deployments to fail immediately at boot instead of crashing under live user load
- Replaced per-request file I/O with in-memory cache, cutting query cost from  $O(N)$  file reads to  $O(1)$  and auto-detecting 30+ dataset parameters via dynamic column introspection
- Layered microbial-community data pipeline into isolated query, transform, and serialization stages, removing inline pandas transforms from FastAPI router

## Projects

### Particle Simulator

Java, Google Truth

- Designed object-oriented 2D grid-based particle physics engine in Java, modeling four distinct particle behaviors (gravity, fluid flow, organic growth, combustion) as independently extensible classes
- Implemented neighbor-map traversal and randomized state-transition algorithms over mutable grid data structure to simulate emergent multi-particle interactions
- Hardened grid-traversal logic against out-of-bounds neighbor access and other boundary-condition edge cases, verifying correctness with Google Truth assertion tests

### Weight Lifting Tracker

React, Node.js, Express, MongoDB, Cloud Run

- Built full-stack workout tracker with Express/MongoDB REST API and Recharts volume-load charts computing sum (weight x reps) over logged sets
- Migrated backend from Render to Google Cloud Run, cutting cold-start latency from over 1 minute to about 5 seconds while shipping React frontend on Vercel

## Technical Skills

**Languages:** Java, Python, TypeScript, JavaScript, SQL

**Frameworks & Libraries:** Spring Boot, Spring Retry, JPA, FastAPI, React, Node.js, Express

**Databases & Infrastructure:** PostgreSQL, DuckDB, MongoDB, Docker, Google Cloud Platform (Pub/Sub, Cloud Run), RESTful API Design